



TRABALHO DE OFICINA DE INTEGRAÇÃO

MazeSync: labirinto físico automatizado

DANILO PIETROBON NETO¹
MONICA PAULA OLIVEIRA MACKERT²

PATO BRANCO - PR
JULHO/2026

¹ Graduando em Engenharia de Computação pela UTFPR-PB.

email: danilopietrobon@alunos.utfpr.edu.br

² Graduada em Engenharia de Computação pela UTFPR-PB.

email: monicamackert@alunos.utfpr.edu.br



RESUMO

Este artigo apresenta o desenvolvimento de um protótipo funcional de um labirinto físico automatizado controlado bilateralmente por meio de hardware dedicado e um aplicativo móvel personalizado. O sistema substitui as abordagens tradicionais baseadas em microcontroladores de arquitetura restrita pelo chip ESP32 de alto desempenho, programado no ecossistema PlatformIO. A inclinação do tabuleiro do labirinto é controlada por dois servo motores configurados nos eixos X e Y. O controle do sistema ocorre por meio de uma lógica de prioridades que integra um joystick analógico físico e um aplicativo Android desenvolvido em Kotlin no Android Studio. A comunicação sem fio é estabelecida configurando o ESP32 como Access Point (AP), recebendo dados de telemetria baseados nos sensores de gravidade e giroscópio do smartphone por meio do protocolo de transporte UDP (User Datagram Protocol). Os resultados obtidos demonstraram baixa latência e alta precisão na manipulação dinâmica e mecânica do tabuleiro, validando a integração de software móvel com sistemas embarcados de baixo custo.

Palavras-chave: ESP32. Android Studio. Kotlin. UDP. Giroscópio. Servomotores.



1. INTRODUÇÃO

O estudo e a implementação de sistemas ciber-físicos, o qual combina processamento computacional, redes de comunicação e processos mecânicos, representam um pilar essencial na formação em Engenharia de Computação. Jogos físicos clássicos de habilidade motora, como o labirinto de inclinação mecânica com esferas metálicas, oferecem um excelente modelo experimental para a aplicação desses conceitos. Tradicionalmente controlados de forma manual por manivelas, esses sistemas podem ser automatizados através de servomotores acionados por microcontroladores.

Contudo, muitos projetos acadêmicos utilizam plataformas limitadas em processamento e sem conectividade nativa, restringindo a experiência de controle a botões ou joysticks com fio. Este projeto propõe a modernização desse cenário através do desenvolvimento de um labirinto inteligente com controle bilateral: físico, via joystick analógico acoplado diretamente à base; e sem fio, utilizando os sensores de movimento internos de um dispositivo móvel.

O sistema foi reprojetoado utilizando o SoC (System on Chip) ESP32, aproveitando sua capacidade de conectividade sem fio integrada e temporizadores robustos. A interface móvel foi desenvolvida em ambiente de produção moderno com a linguagem Kotlin. O objetivo central é analisar a viabilidade técnica e a eficiência de uma comunicação ponto a ponto de baixa latência em tempo real para controle mecânico de precisão, oferecendo uma alternativa robusta e de baixo custo para o ensino prático de sistemas integrados.

2. TRABALHOS RELACIONADOS

- **Sistemas Embarcados com Controle por Fio:** Diversas abordagens acadêmicas utilizam microcontroladores como o Arduino Uno acoplados a joysticks de potenciômetros bidirecionais (ex: KY023). Embora eficazes no controle mecânico básico, sofrem restrições severas de mobilidade espacial devido ao cabeamento físico estrutural.



- **Aplicações de Telemetria com Protocolo TCP:** Projetos que utilizam comunicação sem fio integrada via Wi-Fi frequentemente adotam o protocolo de transporte TCP para o envio de comandos de aplicativos móveis. Apesar de garantir a entrega confiável dos pacotes, a sobrecarga de controle (handshake, retransmissões) gera picos de latência que prejudicam o controle em tempo real de sistemas que necessitam de respostas instantâneas, como o equilíbrio de esferas.
- **Sistemas Móveis de Captura Inercial:** O uso de sensores internos de smartphones (como acelerômetros e giroscópios) é amplamente documentado em jogos digitais mobile. A adaptação desses dados brutos de hardware para a conversão mecânica de inclinação em dispositivos físicos externos de automação representa um campo de estudo dinâmico e em constante evolução no âmbito da Internet das Coisas (IoT).



3. REFERENCIAL TEÓRICO

Integração de Hardware e Software

A convergência entre hardware e software permite a criação de sistemas interativos onde informações do mundo físico são convertidas para dados digitais. No contexto do ecossistema MazeSync, essa integração possibilita a leitura de sensores inerciais e analógicos e a sua tradução em comandos angulares mecânicos.

Protocolo de Transporte UDP

Diferente do protocolo TCP, o UDP opera em modo não-orientado à conexão. Isso significa que os datagramas são enviados diretamente ao destino sem a necessidade de verificação de recebimento ou ordenação de pacotes. Essa ausência de overhead torna o UDP a escolha ideal para sistemas de controle contínuo, onde pacotes antigos perdem a validade e a taxa de atualização instantânea é prioritária.

Sensores Inerciais de Gravidade e Movimento

Dispositivos móveis modernos possuem IMUs (Units de Medição Inercial) integradas. O sensor de gravidade (*Sensor.TYPE_GRAVITY*) realiza uma filtragem de software dos componentes do acelerômetro, isolando a força da gravidade nos três eixos cartesianos (x , y , z). Ao inclinar o celular, os valores gerados representam a aceleração em m/s^2 aplicada sobre cada eixo coordenado.



4. METODOLOGIA

Arquitetura Estrutural do Hardware

O protótipo consiste em uma estrutura física composta por uma base estável, um braço mecânico de suporte articulado em dois eixos e um tabuleiro superior dotado de um labirinto impresso em 3D. A movimentação física é executada por dois servo motores micro (SG90) alimentados por modulação por largura de pulso (PWM) gerada de forma independente através dos timers nativos do ESP32 via biblioteca ESP32Servo. O código de controle foi estruturado com base em uma máquina de estados de prioridades bem definidas rodando em um loop cíclico de 15 *ms*.

Prioridade 1 - Botão SW: O pino digital *PIN_JOY_SW* configurado com resistor pull-up interno monitora continuamente o botão do joystick. Se o botão for pressionado (*LOW*), o sistema executa a função *resetarCentro()*, forçando os servos a ficarem nivelados.

Prioridade 2 - Controle Físico Dinâmico: Se o joystick for retirado de sua zona morta (*JOY_DEADZONE* = 150), os sinais analógicos das portas *PIN_JOY_X* e *PIN_JOY_Y* são convertidos pelo ADC interno (0 a 4095). É calculado um incremento angular dinâmico (*variando de 0.2 a 1.5 graus*), que é somado à posição atual dos servos. Enquanto o joystick estiver em movimento, os comandos sem fio são ignorados.

Prioridade 3 - Controle Móvel: Caso o joystick permaneça estático no centro, o ESP32 passa a processar as requisições de rede. Ele varre o buffer do socket de escuta UDP (*udpPort* = 12345). Se houver pacotes, os dados textuais são processados via função *sscanf()* para extrair os novos ângulos absolutos enviados pelo aplicativo Android.



Para garantir a integridade dos servos e das articulações impressas, todos os ângulos calculados passam obrigatoriamente pela função *limitarAngulo()*, que restringe fisicamente o curso de movimento entre a faixa de segurança de 58 a 118 *graus*.

Para mitigar desvios de montagem, foram criadas as variáveis *SERVO_CENTER_X* e *SERVO_CENTER_Y*, permitindo calibrar de forma independente o nivelamento de cada eixo em 88 *graus* para compensar os desalinhamentos físicos da estrutura.

Conectividade de Rede e Configuração do Access Point (AP)

Para garantir independência de redes locais e roteadores externos, o ESP32 é inicializado programaticamente no modo *Access Point (AP)* de forma isolada. O firmware define uma rede local Wi-Fi restrita com o SSID "*MazeSync – ESP32*" protegida por criptografia *WPA2*. O dispositivo embarcado define para si um IP estático fixo (192.168.4.1) e inicializa o servidor UDP na porta 12345, aguardando pacotes de dados originados do cliente conectado.

Desenvolvimento do Aplicativo Android

O aplicativo de controle móvel foi desenvolvido em ambiente nativo Android Studio utilizando a linguagem Kotlin com suporte à arquitetura moderna de telas cheias (*enableEdgeToEdge*). O fluxo de execução do software consiste na seguinte rotina:

Gerenciamento de Sensores: Durante o ciclo de atividade em primeiro plano (*onResume*), o aplicativo registra um ouvinte (*SensorEventListener*) associado ao *Sensor.TYPE_GRAVITY* configurado com taxa de amostragem otimizada para jogos eletrônicos (*SENSOR_DELAY_GAME*).

Processamento de Dados Inerciais: No evento *onSensorChanged*, os valores brutos de gravidade dos eixos *X* e *Y* são capturados. O aplicativo aplica um fator de escala linear (7.0) adicionado ao centro trigonométrico de 90 graus: $anguloX = 90(gx * 7, 0)$.



Os resultados finais são sanitizados utilizando a função de extensão `.coerceIn(60, 120)`, garantindo a conformidade lógica com os limites mecânicos do hardware.

Transmissão Assíncrona via Rede: Para evitar o congelamento da interface gráfica (*UIThread*) do usuário, sempre que uma alteração angular é detectada, uma linha de execução paralela (*thread*) é gerada. O payload é estruturado como uma String otimizada no formato simples "X,Y", convertida em um array de bytes UTF-8 e encapsulada em um pacote DatagramPacket disparado via DatagramSocket diretamente para o endereço IP do destino (192.168.4.1:12345).



5. RESULTADOS E DISCUSSÕES

Resultados do Sistema Embarcado e Hardware

O ecossistema montado com o PlatformIO garantiu a portabilidade exata e estabilidade das diretivas de compilação do chip esp32doit-devkit-v1. O controle do pulso PWM via hardware utilizando os timers internos do ESP32 eliminou por completo trepidações mecânicas comuns causadas por ruídos de amostragem. A filtragem de software baseada em zonas mortas no código do ADC preveniu movimentos falsos indesejados originados do joystick físico.

Desempenho do Aplicativo Móvel e Comunicação UDP

O sensor de gravidade implementado no Android Studio mostrou-se extremamente responsivo a mudanças sutis de orientação física do smartphone. A interface gráfica do aplicativo atualizou de forma contínua e assíncrona as coordenadas processadas, exibindo na tela os ângulos dinâmicos de transmissão de forma clara.

O uso do protocolo UDP sem conexão eliminou os atrasos severos observados em sistemas semelhantes operando em protocolos orientados a fluxo. A String estruturada separada por vírgulas representou uma carga útil de rede insignificante, permitindo que o processamento interno do ESP32 via sscanf realizasse a extração das coordenadas em microsegundos, resultando em um controle mecânico altamente intuitivo e em tempo real.

Prioridade de Controle

Os testes práticos comprovaram a integridade e eficácia da arquitetura modular de prioridades. Quando o usuário inclinava o celular (transmitindo pacotes contínuos por Wi-Fi) e realizava um toque manual no joystick, o firmware interrompia o processamento da rede imediatamente para obedecer ao periférico físico, retornando ao controle sem fio no exato momento em que o joystick era solto. O acionamento do botão físico de reset interrompeu o fluxo de execução global de forma segura, garantindo o retorno da esfera metálica ao centro do tabuleiro sempre que necessário.

5.1 Demonstração Prática e Resultados Visuais

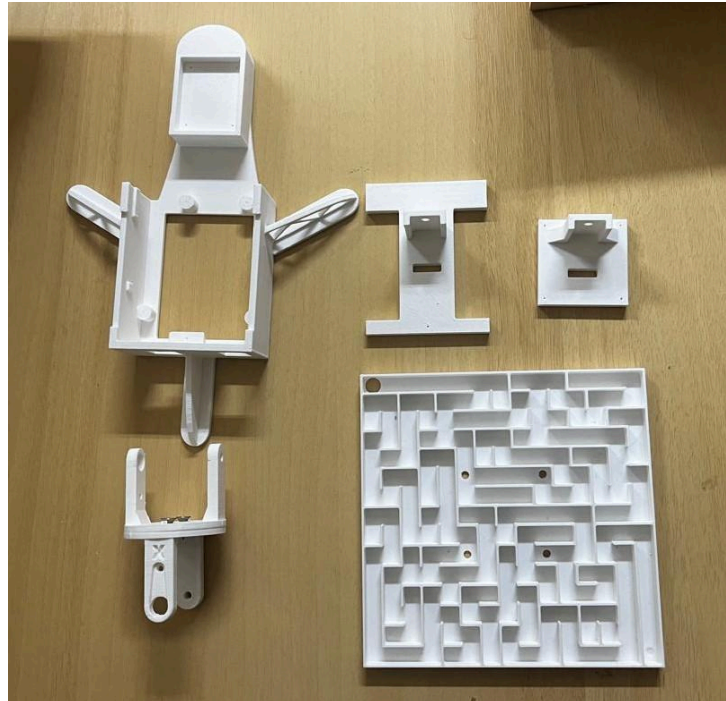


Figura 1: Estrutura física 3D.

Fonte: Autoria própria.

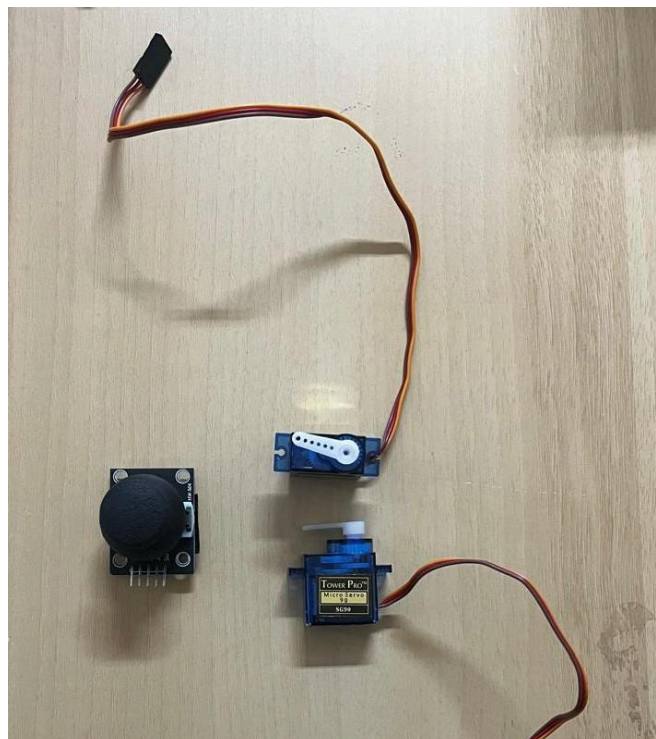


Figura 2: Servo motor SG90 e Joystick KY023

Fonte: Autoria própria.

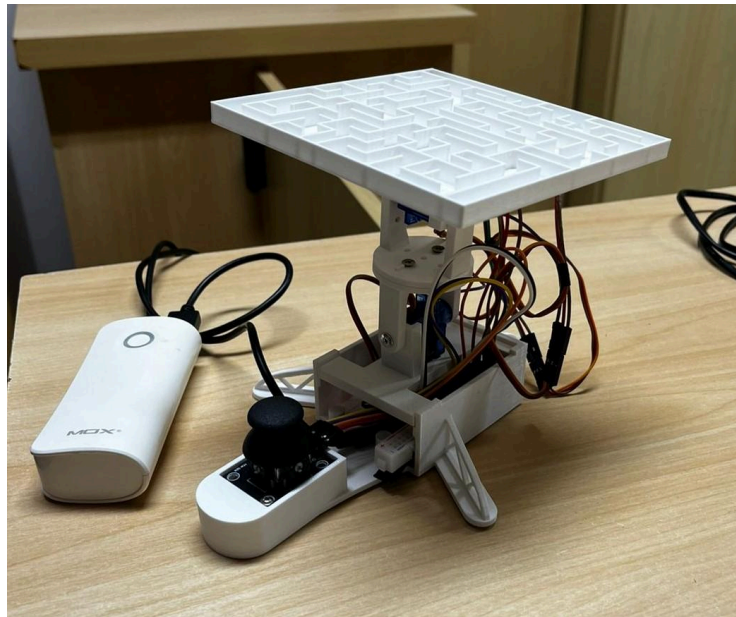


Figura 3: Visual final do labirinto MazeSync.

Fonte: Autoria própria.



Figura 4: Interface do aplicativo Android.

Fonte: Autoria própria.



6. CONSIDERAÇÕES FINAIS

O projeto do sistema de controle dinâmico bilateral de labirinto físico atingiu com sucesso os objetivos estabelecidos na disciplina de Oficina de Integração. A substituição do hardware básico tradicional e de metodologias rudimentares de controle por uma arquitetura combinando o poder computacional do chip ESP32 e o desenvolvimento mobile em Kotlin se provou uma escolha tecnológica superior.

A implementação da rede ponto a ponto local em modo Access Point e o uso estruturado de conexões e soquetes de transporte UDP provaram que é possível desenvolver soluções de controle embarcado sem fio estáveis e rápidas, sem a dependência de infraestruturas locais complexas de rede.

Para melhorias e trabalhos futuros, sugere-se a adição de sensores ópticos nas paredes do labirinto impresso em 3D para cronometrar as partidas, transmitindo o tempo de resolução final de volta para o aplicativo Android a fim de gerar um painel histórico de recordes locais dos usuários.

7. REFERÊNCIAS

ANDROID DEVELOPERS. Sensors Overview. Android Open Source Project, 2026.

Disponível em:

https://developer.android.com/guide/topics/sensors/sensors_overview. Acesso em: 06 jul. 2026.

ARDUINO. Arduino Core AVR: Arduino.h. versão 1.8.6. GitHub, 2024. Disponível em:

<https://github.com/arduino/ArduinoCore-avr/blob/master/cores/arduino/Arduino.h>. Acesso em: 06 jul. 2026.

ESPRESSIF SYSTEMS. Arduino-ESP32 WiFi Library: WiFi.h. GitHub, 2025.

Disponível em:

<https://github.com/espressif/arduino-esp32/blob/master/libraries/WiFi/src/WiFi.h>. Acesso em: 06 jul. 2026.

ESPRESSIF SYSTEMS. Arduino-ESP32 WiFi Library: WiFiUdp.h. GitHub, 2025.

Disponível em:

<https://github.com/espressif/arduino-esp32/blob/master/libraries/WiFi/src/WiFiUdp.h>. Acesso em: 06 jul. 2026.



ESPRESSIF SYSTEMS. ESP-IDF Programming Guide. v5.3. Espressif Systems Documentation, 2026. Disponível em:
<https://docs.espressif.com/projects/esp-idf/en/latest/esp32/>. Acesso em: 06 jul. 2026.

KADNING, J. ESP32Servo Library: ESP32_Servo.h. versão 3.0. GitHub, 2024. Disponível em:
https://github.com/jkb-git/ESP32Servo/blob/master/src/ESP32_Servo.h. Acesso em: 06 jul. 2026.